Chapter 8: Neural Networks — Activations and Loss

Part II — Introduction to Statistical Learning

Omid Safarzadeh

Abstract

A neural network is the linear models of the earlier chapters stacked and bent: each layer is an affine map followed by a nonlinearity, and stacking them lets the model carve curved decision boundaries a single linear layer never could. This chapter assembles the three pieces that make such a network trainable — the *activations* that supply the nonlinearity, the *loss* that scores the output, and *backpropagation*, the chain rule run backwards that delivers every gradient in one sweep. The centrepiece is the softmax cross-entropy gradient, which collapses to the strikingly clean $p - y$, and backprop, which is nothing more mysterious than reusing that derivative layer by layer.

Contents

1 Why a nonlinearity is not optional 1

2 Loss, and the gradient that makes softmax special 2

3 Backpropagation: the chain rule, backwards 2

1 Why a nonlinearity is not optional

A single **layer** takes an input vector x, applies an affine map, and passes the result through a function σ applied coordinate-wise: $a = \sigma(Wx + b)$. The matrix W and bias b are the learnable parameters; σ is the **activation**. The reason σ cannot be the identity is immediate: a composition of affine maps is still affine, so a network of plain linear layers, however deep, collapses to a single linear layer and can only draw straight decision boundaries. The nonlinearity is what buys depth its power.

Definition 1.1 (Common activations)**.** The **sigmoid** $\sigma(z) = 1/(1+e^{-z})$ squashes to $(0, 1)$; **tanh** squashes to $(-1, 1)$; the **ReLU** $\max(0, z)$ passes positives and zeros negatives. For a vector of K output scores (logits) the **softmax** turns them into a probability distribution,

$$\operatorname{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}},$$

which is non-negative and sums to one.

Sigmoid and tanh saturate — their slope vanishes for large $|z|$, which starves the gradient in deep stacks — which is why ReLU, cheap and non-saturating on the positive side, became the default hidden activation. Softmax is reserved for the output of a classifier, where a distribution over classes is exactly what we want. In practice softmax is computed after subtracting $\max_j z_j$ from every logit; this changes nothing mathematically (a constant cancels top and bottom) but keeps e^{z_k} from overflowing.

2 Loss, and the gradient that makes softmax special

Training needs a scalar to minimise. For regression the **mean squared error** $\frac{1}{n}\sum_i(\hat{y}_i - y_i)^2$ is standard. For classification we score the predicted distribution $p = \mathrm{softmax}(z)$ against the one-hot truth y with the **cross-entropy**

$$L = -\sum_{k=1}^{K} y_k \log p_k.$$

Because y is one-hot, L is just $-\log p_{(\text{true class})}$: the loss is small when the network puts mass on the right class and large when it does not. The reason this pairing is used everywhere is the gradient.

Theorem 2.1 (Softmax cross-entropy gradient)**.** Let $p = \mathrm{softmax}(z)$ and $L = -\sum_k y_k \log p_k$ with y a one-hot vector. Then

$$\frac{\partial L}{\partial z_k} = p_k - y_k.$$

Proof. First the softmax's own derivative. With $p_k = e^{z_k}/S$ and $S = \sum_j e^{z_j}$,

$$\frac{\partial p_k}{\partial z_i} = p_k(\delta_{ki} - p_i),$$

where $\delta_{ki} = 1$ if $k = i$ and 0 otherwise (quotient rule: the $k = i$ term contributes p_k, and every term subtracts $p_k p_i$). Now differentiate the loss, using $\partial \log p_k/\partial z_i = (1/p_k)\,\partial p_k/\partial z_i = \delta_{ki} - p_i$:

$$\frac{\partial L}{\partial z_i} = -\sum_k y_k(\delta_{ki} - p_i) = -y_i + p_i \sum_k y_k = p_i - y_i,$$

since $\sum_k y_k = 1$. Renaming $i \to k$ gives the claim. □

The whole softmax Jacobian and the whole cross-entropy derivative cancel into a single subtraction: the gradient at the logits is *the prediction minus the truth*. That is why the pair is the default classification head — the backward pass starts from a quantity you already have.

3 Backpropagation: the chain rule, backwards

A network composes layers, so the loss is a composition of functions and its gradient is a product of Jacobians by the chain rule. **Backpropagation** is the bookkeeping that evaluates that product efficiently: compute the layer outputs forward, then push the gradient backward, each layer turning the gradient of the loss with respect to *its output* into the gradient with respect to its input and its parameters.

For one hidden layer — $z^{(1)} = W_1 x + b_1$, $a = \sigma(z^{(1)})$, $z^{(2)} = W_2 a + b_2$, $p = \mathrm{softmax}(z^{(2)})$ — the backward pass is four lines. Write $\delta^{(2)} = p - y$ (Theorem 2.1). Then

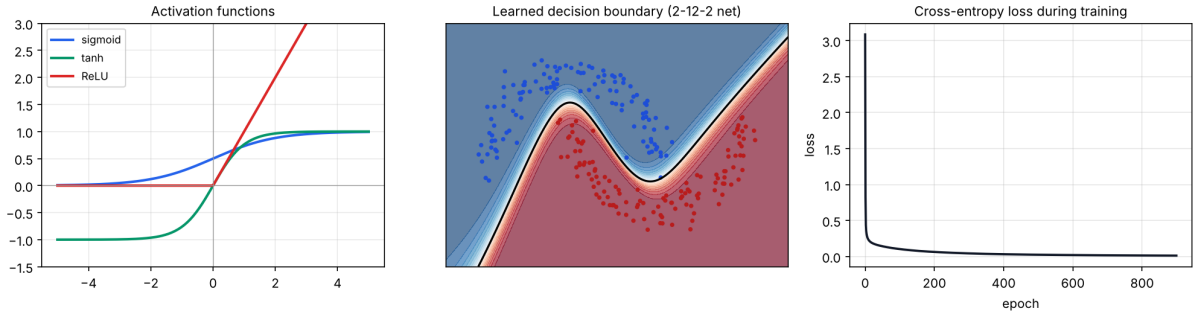
$$\frac{\partial L}{\partial W_2} = \delta^{(2)} a^\top, \quad \delta^{(1)} = \left(W_2^\top \delta^{(2)}\right) \odot \sigma'(z^{(1)}), \quad \frac{\partial L}{\partial W_1} = \delta^{(1)} x^\top,$$

with $\odot$ the element-wise product and the bias gradients equal to the δ's. Each step reuses the δ already computed one layer up — the network is differentiated once, not once per weight, which is the whole reason deep models are trainable.

```python
import numpy as np

def forward_backward(x, y, W1, b1, W2, b2):
    z1 = W1 @ x + b1;  a1 = np.tanh(z1)          # hidden pre-activation + act
    z2 = W2 @ a1 + b2; p = softmax(z2)           # logits -> probabilities
    d2 = p - y                                   # softmax+CE gradient
    dW2 = np.outer(d2, a1);  db2 = d2
    d1  = (W2.T @ d2) * (1 - a1**2)              # back through tanh: s'(z)=1-
        a^2
    dW1 = np.outer(d1, x);   db1 = d1
    return p, (dW1, db1, dW2, db2)
```

Feed those gradients to gradient descent and the network learns. The figure trains exactly this two-layer net on two interleaving half-moons: the activations are on the left, the curved boundary the hidden layer makes possible is in the middle, and the cross-entropy loss falling to near zero is on the right.



Summary

A neural network stacks layers, each an affine map followed by a nonlinear activation; without the nonlinearity the stack collapses to one linear layer. ReLU is the usual hidden activation (cheap, non-saturating), softmax the output of a classifier. Cross-entropy $L = -\sum_k y_k \log p_k$ scores the predicted distribution, and its gradient at the logits is exactly $p - y$ — the whole softmax Jacobian cancels. Backpropagation evaluates the chain-rule product of Jacobians in one backward sweep, each layer mapping the gradient at its output to the gradients at its input and parameters ($\partial L/\partial W = \delta\,(\text{input})^\top$, $\delta^{(\ell)} = (W_{\ell+1}^\top \delta^{(\ell+1)}) \odot \sigma'$), which is what makes deep models trainable.